

SQL Fundamentals

Deep Dive Tutorial — Chapter 01

Relational Databases | DDL | DML | SELECT | JOINS | NULLs | Views

A. Relational Databases	Tables, rows, columns, relationships
B. DDL	CREATE, ALTER, DROP with constraints
C. DML	INSERT, UPDATE, DELETE
D. SELECT Basics	WHERE, ORDER BY, LIMIT
E. Aggregate Functions	COUNT, SUM, AVG, GROUP BY, HAVING
F. JOINS	INNER, LEFT, RIGHT, FULL OUTER
G. Subqueries	Correlated, non-correlated, EXISTS vs IN
H. NULL Handling	IS NULL, COALESCE, three-valued logic
I. Constraints & Referential Integrity	CASCADE DELETE/UPDATE
J. Views	Updatable vs non-updatable views

A What is a Relational Database?

Tables · Rows · Columns · Relationships

Step 1 — Plain English Explanation

A relational database organises data into **tables** (also called relations). Each table represents one real-world entity type — customers, orders, products. A table has **columns** (attributes, e.g. customer_id, name, email) and **rows** (records, one per individual entity instance). Tables are linked by **relationships** — a foreign key in one table references the primary key of another, so you can answer questions that span multiple entities without duplicating data. The whole model is built on **set theory** and **predicate logic** — every query is a mathematical operation on sets of rows.

Step 2 — Real-World Analogy

Think of a relational database as a well-organised office filing system. Each **cabinet drawer** is a table. Each **folder inside** is a row (one customer, one order). Each **labelled field on the form** inside the folder is a column. A sticky note that says 'see drawer ORDERS, folder #1042' is a foreign key — it points you to related data in another drawer without copying all that data again.

Step 3 — Minimal SQL Example

```
-- Two related tables: customers and orders
CREATE TABLE customers (
  customer_id INT PRIMARY KEY,
  name VARCHAR(80) NOT NULL,
  email VARCHAR(120) UNIQUE
);

CREATE TABLE orders (
  order_id INT PRIMARY KEY,
  customer_id INT NOT NULL,          -- foreign key column
  amount DECIMAL(10,2),
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
```

Step 4 — Diagram — Relational Model

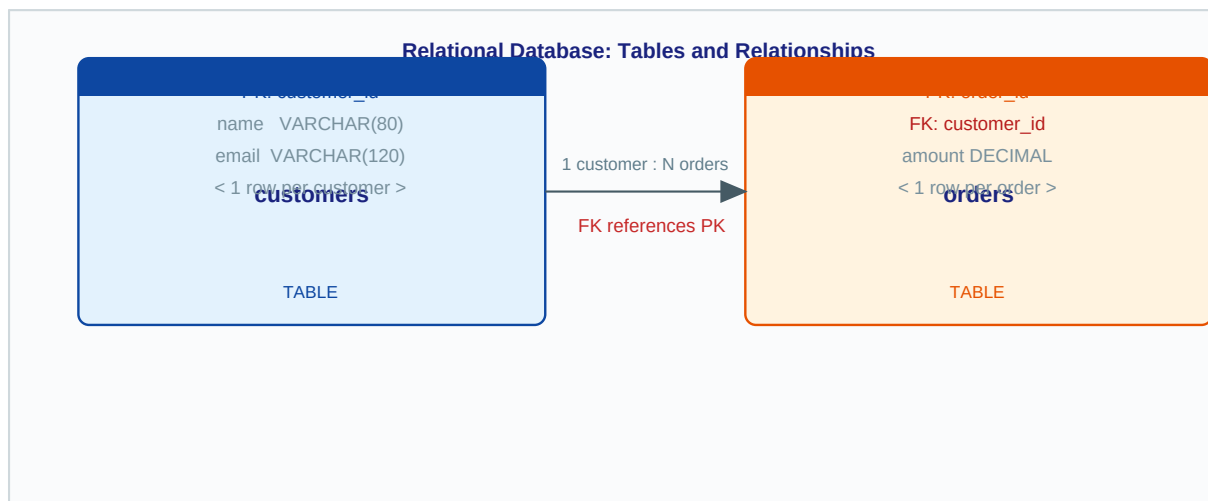


Fig A.1 — customers and orders tables linked by foreign key

Step 5 — Behind the Scenes — What the DB Engine Does

When you create a table the engine allocates **pages** (typically 8 KB or 16 KB blocks) in a data file. Row data is stored inside those pages. The engine maintains a **system catalog** (metadata tables) that records every table name, column name, data type, constraint, and index. When you insert a row the engine finds a page with enough free space, writes the row, and updates the catalog statistics. The **buffer pool** keeps recently used pages in RAM to avoid expensive disk reads.

Step 6 — Realistic Practical Example

An e-commerce platform has: **customers**, **products**, **orders**, and **order_items** (a junction/bridge table). Instead of repeating the customer name in every order row, you store `customer_id`. This eliminates duplication, saves disk space, and means a name change only requires updating one row in customers — not thousands of order rows. This is called **normalisation**.

Step 7 — Common Mistakes and Misconceptions

- Thinking a 'database' and a 'table' are the same thing. A database contains many tables.
 - Confusing rows with columns. Columns define structure; rows hold data.
 - Storing repeated data (e.g. customer name in every order row). Use a foreign key instead.
 - Assuming row order is guaranteed. SQL tables are unordered sets — always use `ORDER BY` for sorted results.
 - Not defining a `PRIMARY KEY`. Without one, the engine cannot uniquely identify a row.
-

B

DDL — Data Definition Language

CREATE TABLE · ALTER TABLE · DROP TABLE · Constraints

Step 1 — Plain English Explanation

DDL (Data Definition Language) is the subset of SQL used to define and modify the **structure** of the database — tables, columns, constraints, and indexes. Unlike DML (which changes data), DDL changes the schema itself. The three core DDL statements are **CREATE TABLE** (build a new table), **ALTER TABLE** (modify an existing table), and **DROP TABLE** (permanently remove a table and all its data). Constraints — PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK — are also declared in DDL and are enforced by the engine on every DML operation.

Step 2 — Real-World Analogy

DDL is like designing and printing the blank form used in an office. CREATE TABLE prints the form (defines which fields exist). ALTER TABLE is like adding a new field to an existing form. DROP TABLE is shredding all copies. Constraints are the validation rules printed on the form: 'Employee ID is required', 'Email must be unique', 'Department must exist in the Departments list'.

Step 3 — Minimal SQL Example

```
-- CREATE TABLE with all major constraint types
CREATE TABLE employees (
  emp_id      INT          PRIMARY KEY,          -- unique, not null
  dept_id     INT          NOT NULL,
  name        VARCHAR(100) NOT NULL,
  email       VARCHAR(150) UNIQUE,              -- no duplicates
  salary      DECIMAL(10,2) CHECK (salary > 0), -- value rule
  hire_date   DATE         DEFAULT CURRENT_DATE,
  FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);

-- ALTER TABLE: add a column
ALTER TABLE employees ADD COLUMN phone VARCHAR(20);

-- DROP TABLE (permanent!)
DROP TABLE employees;
```

Step 4 — Diagram — DDL Constraint Map

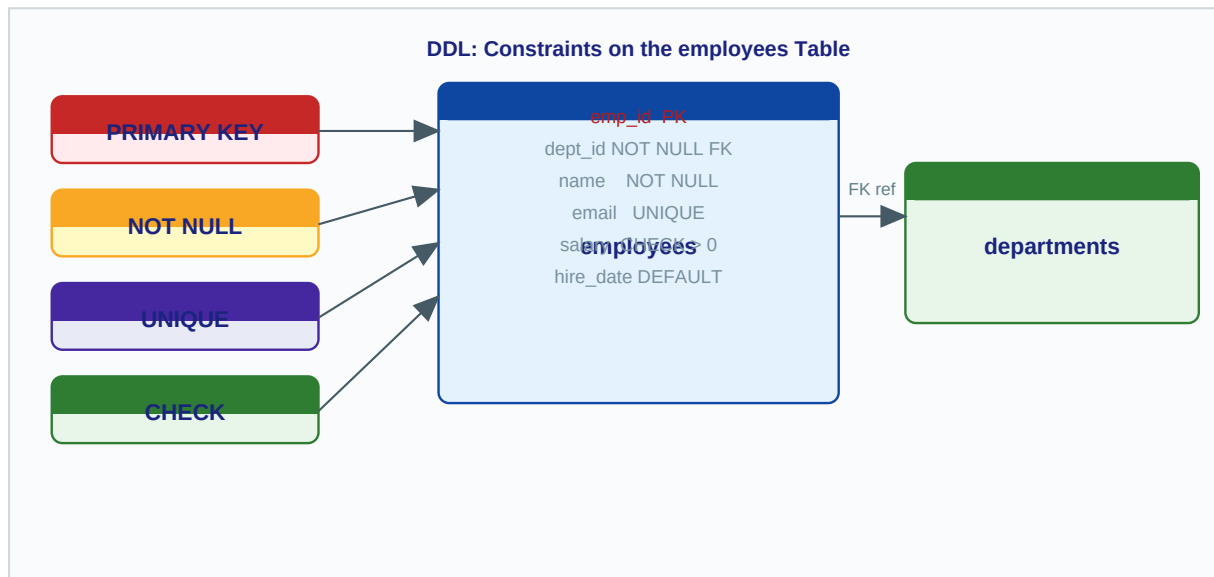


Fig B.1 — employees table with all constraint types annotated

Step 5 — Behind the Scenes

When you run `CREATE TABLE` the engine writes a row to its internal system catalog (e.g. `pg_class` in PostgreSQL, `information_schema.tables`). A **PRIMARY KEY** automatically creates a **B-tree index** on that column — this is why PK lookups are fast. A **UNIQUE** constraint also creates a B-tree index. A **FOREIGN KEY** constraint causes the engine to check the referenced table on every `INSERT/UPDATE` to the child table and on every `DELETE/UPDATE` to the parent table. `ALTER TABLE` on a large table can be expensive because some changes (e.g. adding a `NOT NULL` column without a default) require rewriting every row.

Step 6 — Realistic Practical Example

```

-- Real schema: e-commerce product catalog
CREATE TABLE categories (
  cat_id INT PRIMARY KEY,
  cat_name VARCHAR(60) NOT NULL UNIQUE
);

CREATE TABLE products (
  product_id INT PRIMARY KEY,
  cat_id INT NOT NULL,
  sku VARCHAR(30) UNIQUE NOT NULL,
  price DECIMAL(10,2) NOT NULL CHECK (price >= 0),
  stock_qty INT DEFAULT 0 CHECK (stock_qty >= 0),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (cat_id) REFERENCES categories(cat_id)
);
  
```

Step 7 — Common Mistakes and Misconceptions

- Using `VARCHAR(MAX)` everywhere. Always choose the smallest appropriate data type.
- Forgetting `NOT NULL` on columns that should never be empty. `NULL` creeps in silently.
- `DROP TABLE` without a backup. It is immediate and permanent in most engines.
- Thinking `ALTER TABLE` is free. On large tables, adding/changing columns can lock the table for minutes.
- Duplicate `UNIQUE + PRIMARY KEY` — `PRIMARY KEY` already implies `UNIQUE`.

- FOREIGN KEY without an index on the referencing column — this causes slow DELETE on the parent.
-

C

DML — Data Manipulation Language

INSERT · UPDATE · DELETE

Step 1 — Plain English Explanation

DML (Data Manipulation Language) is the subset of SQL used to add, change, and remove rows inside tables. The three statements are: **INSERT** (add new rows), **UPDATE** (change existing rows), and **DELETE** (remove rows). DML operates within transactions — changes can be rolled back with ROLLBACK if something goes wrong, or permanently committed with COMMIT. Unlike DDL, DML only touches data, not the schema.

Step 2 — Real-World Analogy

DML is the day-to-day work of filling in, correcting, and removing records in the office filing system. INSERT is adding a new folder. UPDATE is whiting out a wrong value and writing the correct one. DELETE is pulling the folder out and throwing it away. All changes happen in pencil until you say COMMIT — only then are they permanent.

Step 3 — Minimal SQL Example

```
-- INSERT: single row
INSERT INTO customers (customer_id, name, email)
VALUES (1, 'Alice Smith', 'alice@example.com');

-- INSERT: multiple rows
INSERT INTO customers (customer_id, name, email) VALUES
  (2, 'Bob Jones', 'bob@example.com'),
  (3, 'Carol Lee', 'carol@example.com');

-- UPDATE: give all customers in city 'London' a discount flag
UPDATE customers
SET    discount = TRUE
WHERE  city = 'London';

-- DELETE: remove inactive customers
DELETE FROM customers
WHERE  last_login < '2023-01-01';
```

Step 4 — Diagram — DML Flow

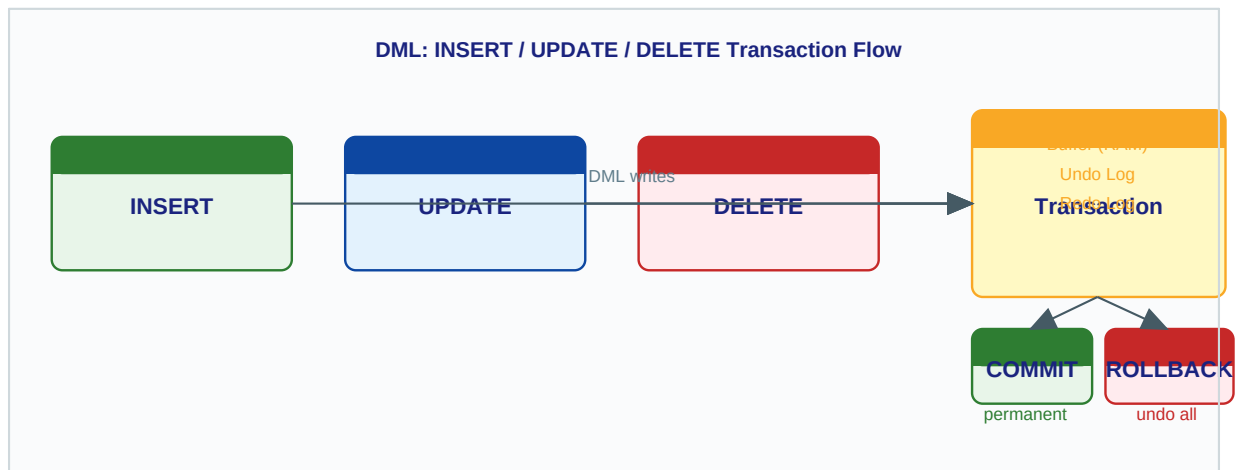


Fig C.1 — DML statements write to a transaction buffer; COMMIT or ROLLBACK decides the outcome

Step 5 — Behind the Scenes

Every DML statement in a transaction writes to the engine's **undo log** (stores the old values so ROLLBACK can restore them) and **redo log / WAL (Write-Ahead Log)** (stores the new values so changes survive a crash). The actual data pages are updated in the **buffer pool** (RAM). Pages are written to disk asynchronously by the background checkpoint process. On COMMIT, the engine flushes the redo log to disk and marks the transaction as committed. On crash recovery, the engine replays the redo log to reconstruct committed changes and uses the undo log to roll back uncommitted ones.

Step 6 — Realistic Practical Example

```
-- Realistic: process an order placement atomically
BEGIN;

INSERT INTO orders (order_id, customer_id, total)
VALUES (5001, 42, 299.99);

INSERT INTO order_items (order_id, product_id, qty, unit_price)
VALUES (5001, 101, 2, 99.99),
       (5001, 205, 1, 100.01);

UPDATE products
SET    stock_qty = stock_qty - 2
WHERE  product_id = 101;

UPDATE products
SET    stock_qty = stock_qty - 1
WHERE  product_id = 205;

COMMIT; -- all 4 statements succeed together or none do
```

Step 7 — Common Mistakes and Misconceptions

- UPDATE or DELETE without a WHERE clause — updates/deletes EVERY row. Always double-check.
 - Not using transactions for multi-statement operations — partial failures leave data inconsistent.
 - INSERT without specifying column names — breaks silently when columns are reordered.
 - Thinking DELETE frees disk space immediately. Most engines mark rows as deleted; space is reclaimed later (VACUUM/OPTIMIZE).
 - Using UPDATE to 'move' data — UPDATE changes values in place; it does not move rows between tables.
-

D SELECT Basics

WHERE · ORDER BY · LIMIT · Execution Order

Step 1 — Plain English Explanation

SELECT is the SQL statement for querying data. You specify **which columns** to return, **which table(s)** to read from, **filtering conditions** (WHERE), **sort order** (ORDER BY), and optionally **how many rows** to return (LIMIT). The crucial insight is that the **written order** of clauses (SELECT...FROM...WHERE...) is NOT the order the engine executes them. The **logical execution order** is: FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT.

Step 2 — Real-World Analogy

SELECT is like asking a librarian a precise question: 'From the SCIENCE section (FROM), find all books published after 2020 (WHERE), sorted alphabetically by title (ORDER BY), and show me just the first 10 results (LIMIT). For each book, tell me only the title and author (SELECT columns).' The librarian doesn't show you titles first and then fetch books — they go to the section first, filter by date, sort, then give you the first 10.

Step 3 — Minimal SQL Example

```
-- Basic SELECT with all clauses
SELECT  name,
        email,
        salary
FROM    employees
WHERE   salary > 50000
        AND dept_id = 3
ORDER BY salary DESC
LIMIT  10;

-- Wildcard (avoid in production code)
SELECT * FROM employees WHERE emp_id = 42;

-- Aliases for readability
SELECT  emp_id AS id,
        name   AS full_name
FROM    employees;
```

Step 4 — Diagram — Logical Execution Order



Fig D.1 — The 8-step logical execution order of a SELECT statement

Step 5 — Behind the Scenes

The query engine first parses the SQL into an **Abstract Syntax Tree (AST)**, then the **query planner / optimiser** rewrites it into the most efficient **execution plan**. The planner considers available indexes, table statistics (row counts, value distributions), and join strategies. It produces a plan like: 'Index seek on employees.salary, filter dept_id=3, sort top-10'. The plan can be inspected with EXPLAIN or EXPLAIN ANALYZE. WHERE clauses run before SELECT, so you **cannot** use a column alias defined in SELECT inside a WHERE clause — the alias does not exist yet at WHERE execution time.

Step 6 — Realistic Practical Example

```
-- Find top 5 highest-paid employees in HR dept, hired after 2020
SELECT  e.emp_id,
        e.name,
        e.salary,
        e.hire_date
FROM    employees AS e
WHERE   e.dept_id = (
        SELECT dept_id FROM departments WHERE dept_name = 'HR'
      )
  AND   e.hire_date > '2020-01-01'
ORDER BY e.salary DESC
LIMIT 5;
```

Step 7 — Common Mistakes and Misconceptions

- Using SELECT * in application code — schema changes silently break your app.
- Referencing a SELECT alias in WHERE: SELECT salary*1.1 AS bonus FROM ... WHERE bonus > 5000 — FAILS because WHERE runs before SELECT.
- Assuming ORDER BY without LIMIT is free — sorting large result sets is expensive.
- LIMIT without ORDER BY gives unpredictable rows — the engine returns any N rows it finds first.
- WHERE column = NULL — always wrong. Use WHERE column IS NULL (see Section H).

E Aggregate Functions

COUNT · SUM · AVG · MIN · MAX · GROUP BY · HAVING

Step 1 — Plain English Explanation

Aggregate functions collapse multiple rows into a single result. **COUNT(*)** counts rows; **SUM** adds values; **AVG** computes the mean; **MIN/MAX** find extremes. Used alone they produce one row for the whole table. Paired with **GROUP BY**, they produce one row per group (e.g. one row per department). **HAVING** filters groups after aggregation — it is the WHERE clause for groups, not individual rows.

Step 2 — Real-World Analogy

Imagine a spreadsheet of sales transactions. **GROUP BY** department is like inserting a subtotal row for each department. **SUM(sales)** totals each subtotal row. **HAVING SUM(sales) > 100000** hides departments with low totals — like an Excel AutoFilter on the subtotal column. **WHERE** filters individual transaction rows before grouping; **HAVING** filters the subtotal rows after.

Step 3 — Minimal SQL Example

```
-- Total employees and average salary per department
SELECT  dept_id,
        COUNT(*)      AS headcount,
        AVG(salary)    AS avg_salary,
        MIN(salary)    AS min_salary,
        MAX(salary)    AS max_salary,
        SUM(salary)    AS total_payroll
FROM    employees
WHERE   hire_date > '2019-01-01'  -- filter rows BEFORE grouping
GROUP BY dept_id
HAVING  COUNT(*) > 5              -- filter groups AFTER aggregation
ORDER BY avg_salary DESC;
```

Step 4 — Diagram — GROUP BY Mechanics

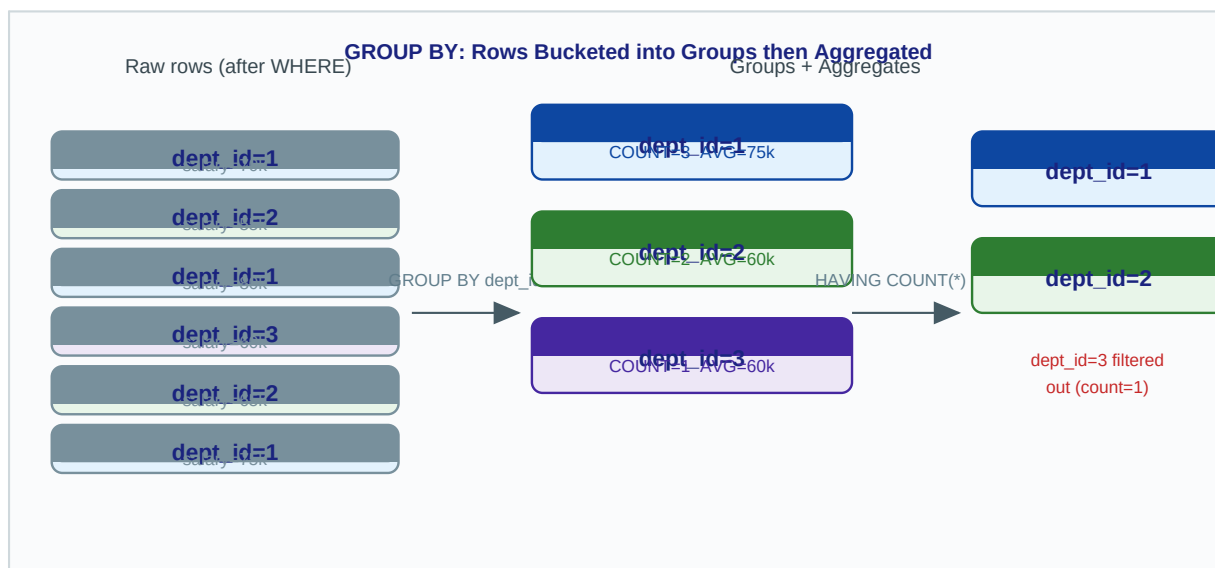


Fig E.1 — GROUP BY buckets rows; HAVING filters buckets by aggregate result

Step 5 — Behind the Scenes

After the WHERE filter, the engine sorts (or hash-buckets) rows by the GROUP BY columns. Each unique combination of GROUP BY values forms a group. The aggregate function is then computed for each group. **Important engine rule:** every column in the SELECT list must either appear in GROUP BY or be inside an aggregate function. This is because after grouping, a group represents multiple rows — referencing a non-grouped, non-aggregated column would be ambiguous (which row's value should be returned?). Some engines (MySQL in lenient mode) allow this but return an arbitrary value — a well-known trap.

Step 6 — Realistic Practical Example

```
-- Monthly revenue and order count per product category
SELECT  c.cat_name,
        DATE_FORMAT(o.order_date, '%Y-%m') AS month,
        COUNT(DISTINCT o.order_id)          AS orders,
        SUM(oi.qty * oi.unit_price)         AS revenue
FROM    order_items AS oi
JOIN    orders      AS o  ON o.order_id = oi.order_id
JOIN    products    AS p  ON p.product_id = oi.product_id
JOIN    categories  AS c  ON c.cat_id   = p.cat_id
WHERE   o.order_date >= '2024-01-01'
GROUP BY c.cat_name, DATE_FORMAT(o.order_date, '%Y-%m')
HAVING  SUM(oi.qty * oi.unit_price) > 10000
ORDER BY month, revenue DESC;
```

Step 7 — Common Mistakes and Misconceptions

- WHERE vs HAVING confusion — WHERE filters rows (before grouping); HAVING filters groups (after aggregation).
 - COUNT(column) vs COUNT(*) — COUNT(column) ignores NULLs; COUNT(*) counts all rows including NULLs.
 - Selecting a non-grouped column: SELECT name, dept_id, AVG(salary) FROM ... GROUP BY dept_id — 'name' is ambiguous, will error in strict mode.
 - AVG on integer columns may lose decimal precision in some engines — cast to DECIMAL first.
 - Using HAVING instead of WHERE for row-level filters — HAVING runs after aggregation, so it is slower for row filters.
-

F JOINS

INNER · LEFT · RIGHT · FULL OUTER · Engine Internals

Step 1 — Plain English Explanation

A JOIN combines rows from two (or more) tables based on a related column. The four main types: **INNER JOIN** — only rows with a match in BOTH tables; **LEFT JOIN** — all rows from the left table, matched rows from the right (NULLs for unmatched right); **RIGHT JOIN** — opposite of LEFT JOIN; **FULL OUTER JOIN** — all rows from both tables, NULLs where there is no match. The join condition (usually ON t1.id = t2.fk) specifies which column pairs to match.

Step 2 — Real-World Analogy

Two lists: List A = all employees. List B = all department records. INNER JOIN = employees who have a matching department (unassigned employees are excluded). LEFT JOIN = all employees, showing department info where it exists, blank otherwise. RIGHT JOIN = all departments, showing their employees (empty departments also appear). FULL OUTER JOIN = every employee and every department, with blanks on either side where there is no match.

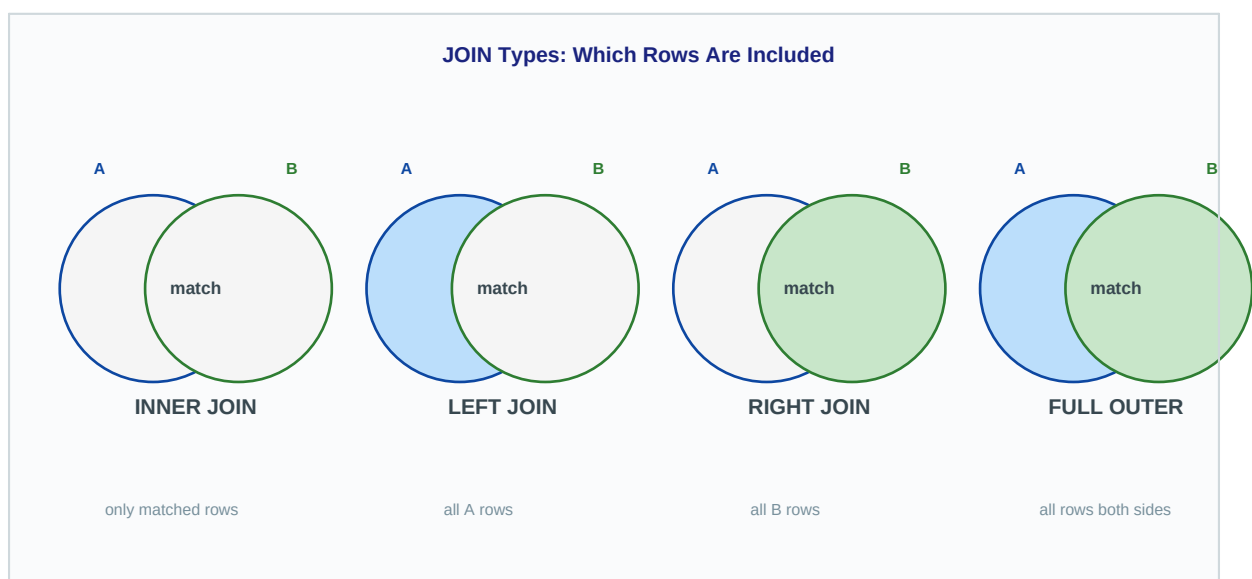
Step 3 — Minimal SQL Example

```
-- INNER JOIN: only employees WITH a department
SELECT e.name, d.dept_name
FROM   employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;

-- LEFT JOIN: all employees, NULL dept_name for unassigned
SELECT e.name, d.dept_name
FROM   employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id;

-- FULL OUTER JOIN: all employees + all departments
SELECT e.name, d.dept_name
FROM   employees e
FULL OUTER JOIN departments d ON e.dept_id = d.dept_id;
```

Step 4 — Diagram — JOIN Venn Diagrams



Step 5 — Behind the Scenes — Join Algorithms

The query optimiser chooses one of three join algorithms based on table sizes and available indexes:

- **Nested Loop Join** — for each row in the outer table, scan the inner table for matches. $O(n*m)$. Good for small inner tables or when an index exists on the join column.
- **Hash Join** — build a hash table from the smaller table, then probe it with each row from the larger table. $O(n+m)$. Good for large tables with no useful index.
- **Merge Join** — both tables must be sorted on the join column (or already indexed). Merges them like a zipper. $O(n+m)$ but requires a sort step if not pre-sorted.

The optimiser picks based on cost estimates from table statistics. You can force a join algorithm with hints in some engines (e.g. MySQL: `STRAIGHT_JOIN`).

Step 6 — Realistic Practical Example

```
-- Find customers who have NEVER placed an order (LEFT JOIN anti-pattern)
SELECT  c.customer_id,
        c.name,
        c.email
FROM    customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE   o.order_id IS NULL      -- no match in orders = never ordered
ORDER BY c.name;

-- Multi-table JOIN: order details with customer and product names
SELECT  o.order_id,
        c.name      AS customer,
        p.sku       AS product,
        oi.qty,
        oi.unit_price
FROM    orders o
JOIN    customers  c ON c.customer_id = o.customer_id
JOIN    order_items oi ON oi.order_id  = o.order_id
JOIN    products   p ON p.product_id  = oi.product_id;
```

Step 7 — Common Mistakes and Misconceptions

- Forgetting the ON condition — a JOIN without ON produces a CROSS JOIN (cartesian product: every row with every row).
 - Confusing LEFT JOIN result: NULL in the right-side column means 'no match', not that the column value is NULL in the right table.
 - FULL OUTER JOIN is not supported in MySQL — use UNION of LEFT JOIN and RIGHT JOIN.
 - Joining on columns with different data types silently disables index usage in some engines.
 - Many JOINS on large tables without indexes on join columns cause full table scans — extremely slow.
-

G

Subqueries

Correlated · Non-correlated · EXISTS vs IN

Step 1 — Plain English Explanation

A subquery is a `SELECT` statement nested inside another SQL statement. A **non-correlated subquery** runs once and returns a result that the outer query uses — it is independent of the outer query. A **correlated subquery** runs once per row of the outer query because it references a column from the outer query. **EXISTS** tests whether a subquery returns any rows at all (stops at the first match). **IN** tests whether a value matches any value in a list returned by a subquery.

Step 2 — Real-World Analogy

Non-correlated: 'Find employees who earn more than the *company average salary*'. The average is calculated once. Correlated: 'For each employee, find their *department's* average salary' — each employee triggers a separate average calculation for their own department. EXISTS is like checking 'does this employee have at least one order?' — you stop searching as soon as you find the first one.

Step 3 — Minimal SQL Example

```
-- Non-correlated: employees earning above company average
SELECT name, salary
FROM   employees
WHERE  salary > (SELECT AVG(salary) FROM employees);

-- Correlated: employees earning above their OWN dept average
SELECT e.name, e.salary, e.dept_id
FROM   employees e
WHERE  e.salary > (
    SELECT AVG(e2.salary)
    FROM   employees e2
    WHERE  e2.dept_id = e.dept_id  -- references outer query
);

-- EXISTS: customers who placed at least one order
SELECT c.name
FROM   customers c
WHERE  EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
);
```

Step 4 — Diagram — Correlated vs Non-correlated

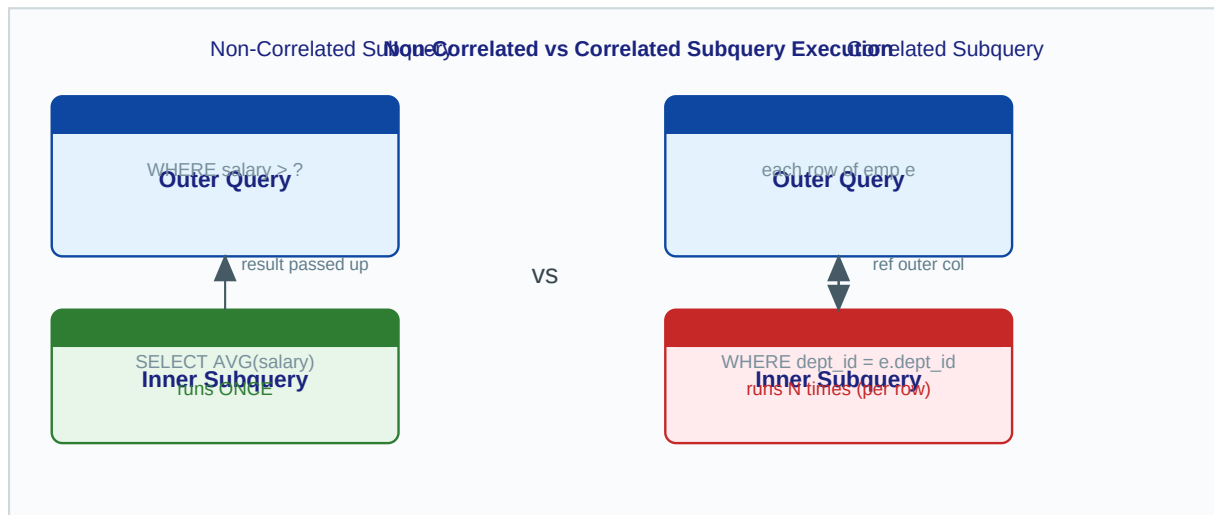


Fig G.1 — Non-correlated runs once; correlated runs once per outer row

Step 5 — Behind the Scenes

The optimiser often rewrites subqueries into JOINS internally for performance. A correlated subquery that the optimiser cannot rewrite runs N times (once per outer row) which is $O(n*m)$ — slow on large tables. EXISTS with a correlated subquery short-circuits after the first match, making it faster than IN for large lists. IN vs EXISTS behaviour with NULLs also differs: if the IN subquery returns any NULL, the overall result can be UNKNOWN (not FALSE) — a subtle trap covered in Section H.

Step 6 — Realistic Practical Example

```
-- NOT EXISTS anti-join: customers with no orders this year
SELECT c.customer_id, c.name
FROM   customers c
WHERE  NOT EXISTS (
    SELECT 1
    FROM    orders o
    WHERE   o.customer_id = c.customer_id
    AND     o.order_date >= '2024-01-01'
);

-- IN with subquery: products in top-selling categories
SELECT product_id, sku, price
FROM   products
WHERE  cat_id IN (
    SELECT cat_id
    FROM   order_items oi
    JOIN   products p ON p.product_id = oi.product_id
    GROUP BY cat_id
    HAVING SUM(oi.qty) > 500
);
```

Step 7 — Common Mistakes and Misconceptions

- Using a correlated subquery in SELECT for every row: `SELECT name, (SELECT dept_name FROM ... WHERE ...)` — runs N times; use a JOIN instead.
- NOT IN with a subquery that can return NULL — if the subquery returns even one NULL, NOT IN returns no rows. Use NOT EXISTS instead.
- Forgetting that IN is equivalent to `= ANY(...)`; if performance is critical, a JOIN is usually faster.
- Nesting subqueries 3+ levels deep — hard to read and optimise; refactor with CTEs (WITH clause).

H NULL Handling

IS NULL · COALESCE · NULLIF · Three-Valued Logic

Step 1 — Plain English Explanation

NULL in SQL means 'unknown' or 'not applicable' — it is not zero, not an empty string, not false. NULL is a missing value. Because of this, SQL uses **three-valued logic**: every comparison evaluates to TRUE, FALSE, or **UNKNOWN**. Any arithmetic or comparison involving NULL produces UNKNOWN (or NULL). WHERE and JOIN conditions only pass rows where the condition is TRUE — UNKNOWN rows are silently dropped. Use **IS NULL** / **IS NOT NULL** to test for NULL. **COALESCE** returns the first non-NULL value in a list. **NULLIF** returns NULL if two values are equal.

Step 2 — Real-World Analogy

NULL is like an answer of 'I don't know' on a form. If you ask 'Is this person's age greater than 30?' and the age field is blank (unknown), the honest answer is 'I don't know' — not yes, not no. SQL treats that 'I don't know' as UNKNOWN, and UNKNOWN rows are excluded from results. This is why **NULL != NULL** — asking 'Is Unknown equal to Unknown?' also returns 'I don't know'.

Step 3 — Minimal SQL Example

```
-- Wrong: this returns NO rows, even where phone IS NULL
SELECT * FROM employees WHERE phone = NULL;

-- Correct: use IS NULL
SELECT * FROM employees WHERE phone IS NULL;

-- COALESCE: use 'N/A' if phone is NULL
SELECT name, COALESCE(phone, 'N/A') AS phone_display
FROM employees;

-- NULLIF: treat empty string as NULL
SELECT name, NULLIF(phone, '') AS phone
FROM employees;

-- NULL in aggregates: COUNT ignores NULLs
SELECT COUNT(phone) AS with_phone, -- ignores NULLs
       COUNT(*)     AS total_rows  -- counts all rows
FROM employees;
```

Step 4 — Diagram — Three-Valued Logic Truth Table

NULL: Three-Valued Logic and Common Traps					
A	B	A AND B	A OR B	NOT A	
TRUE	TRUE	TRUE	TRUE	FALSE	
TRUE	FALSE	FALSE	TRUE	FALSE	
TRUE	NULL	UNKNOWN	TRUE	FALSE	
FALSE	NULL	FALSE	UNKNOWN	TRUE	
NULL	NULL	UNKNOWN	UNKNOWN	UNKNOWN	

NULL = NULL evaluates to UNKNOWN (not TRUE!)	NULL IS NULL evaluates to TRUE -- use this form
WHERE salary > NULL UNKNOWN -- row is excluded silently	WHERE salary IS NOT NULL correct pattern

Fig H.1 — Three-valued logic truth table and NULL trap examples

Step 5 — Behind the Scenes

NULL is stored in a special per-row **null bitmap** — a bitmask at the start of each row that records which columns are NULL. No actual value is stored for NULL columns. In a B-tree index, NULLs are indexed in PostgreSQL (at the beginning or end) but NOT indexed in Oracle (NULLs are excluded from B-tree indexes). This means WHERE col IS NULL does a full table scan in Oracle unless a special index is created. JOIN conditions with NULLs never match — two NULL foreign keys do not join together because NULL != NULL in three-valued logic.

Step 6 — Realistic Practical Example

```
-- Report: show 0 if no sales, not NULL
SELECT  e.name,
        COALESCE(SUM(s.amount), 0) AS total_sales
FROM    employees e
LEFT JOIN sales s ON s.emp_id = e.emp_id
GROUP BY e.emp_id, e.name;

-- Safe division: avoid divide-by-zero using NULLIF
SELECT  dept_id,
        total_payroll / NULLIF(headcount, 0) AS avg_salary
FROM    dept_summary;

-- Find rows where optional field was never filled in
SELECT name FROM employees
WHERE  manager_id IS NULL;  -- top-level managers
```

Step 7 — Common Mistakes and Misconceptions

- WHERE col = NULL — always returns 0 rows. Always use IS NULL.
- NOT IN subquery returning NULLs — NOT IN (1, 2, NULL) returns UNKNOWN for everything. Use NOT EXISTS.
- Believing NULLs are excluded from aggregates — AVG, SUM, COUNT(col) all ignore NULLs; only COUNT(*) includes them.
- NULL in a UNIQUE constraint — most databases allow multiple NULLs in a UNIQUE column (because NULL != NULL).

- NULL in string concatenation: 'Hello' || NULL = NULL in SQL standard — use COALESCE to guard.
-

I Constraints & Referential Integrity

CASCADE DELETE · CASCADE UPDATE · Constraint Types

Step 1 — Plain English Explanation

Referential integrity means the database engine enforces that foreign key values always point to an existing row in the referenced table. You cannot insert an order with a `customer_id` that does not exist. **CASCADE DELETE** automatically deletes child rows when the parent row is deleted. **CASCADE UPDATE** propagates a primary key change to all foreign key references. Alternatives are **SET NULL** (set FK to NULL on parent delete), **SET DEFAULT** (set FK to its default value), and **RESTRICT / NO ACTION** (block the parent DELETE/UPDATE if child rows exist).

Step 2 — Real-World Analogy

Referential integrity is the rule that a company cannot delete an employee file while that employee is still listed as the manager of active projects. **CASCADE DELETE** is like a policy: 'When we delete a department, automatically fire all its employees'. **SET NULL** is: 'When the department is dissolved, leave the employees but set their department field to blank'. **RESTRICT** is: 'Block the department deletion until all employees are reassigned first'.

Step 3 — Minimal SQL Example

```
CREATE TABLE departments (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(80) NOT NULL  
);  
  
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    dept_id INT,  
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)  
        ON DELETE CASCADE -- delete employee when dept deleted  
        ON UPDATE CASCADE -- update FK when dept_id changes  
);  
  
-- Delete dept 10: automatically deletes all employees in dept 10  
DELETE FROM departments WHERE dept_id = 10;  
  
-- With SET NULL: employee stays, dept_id becomes NULL  
-- FOREIGN KEY (dept_id) REFERENCES departments(dept_id) ON DELETE SET NULL
```

Step 4 — Diagram — Referential Integrity Actions

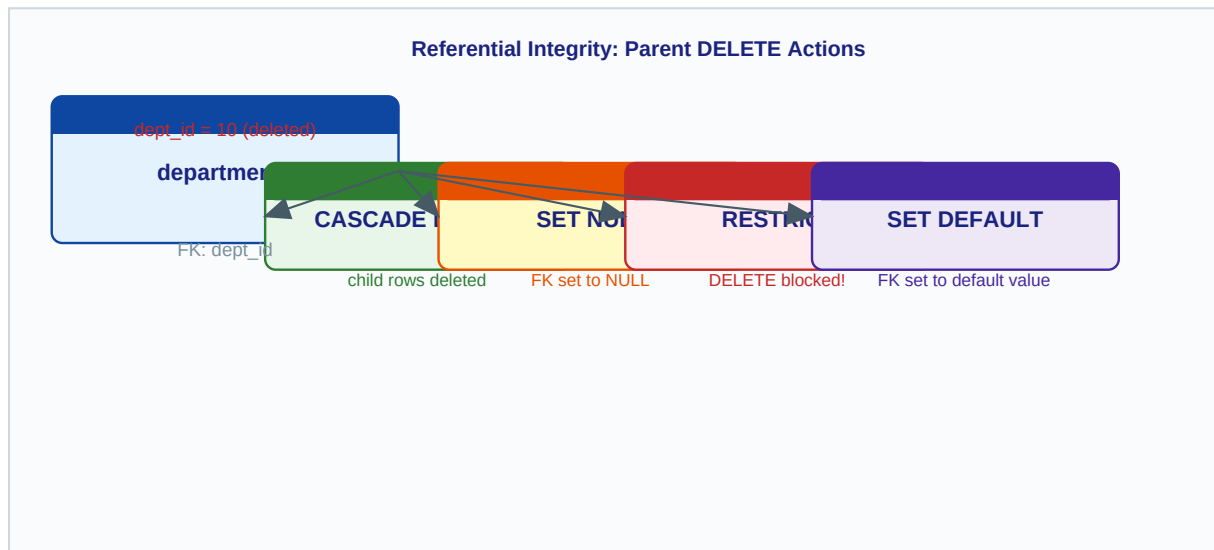


Fig 1.1 — Four referential integrity actions when a parent row is deleted

Step 5 — Behind the Scenes

When you DELETE a parent row, the engine checks the FK constraint index to find child rows. For CASCADE DELETE it issues internal DELETE statements for those child rows (recursively — grandchildren are also cascaded). For RESTRICT / NO ACTION the engine raises an error before the delete completes. Note: **NO ACTION** is deferred (checked at transaction end in some engines) while RESTRICT is immediate. CASCADE is powerful but dangerous — an accidental delete of a top-level row can wipe thousands of child rows across multiple tables. Always use CASCADE with careful application logic.

Step 6 — Realistic Practical Example

```
-- E-commerce: order_items cascade-deleted with their order
CREATE TABLE orders (
  order_id    INT PRIMARY KEY,
  customer_id INT NOT NULL,
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
    ON DELETE RESTRICT  -- cannot delete customer with orders
);

CREATE TABLE order_items (
  item_id    INT PRIMARY KEY,
  order_id   INT NOT NULL,
  product_id INT NOT NULL,
  qty        INT NOT NULL DEFAULT 1,
  FOREIGN KEY (order_id) REFERENCES orders(order_id)
    ON DELETE CASCADE,  -- delete items when order deleted
  FOREIGN KEY (product_id) REFERENCES products(product_id)
    ON DELETE RESTRICT  -- cannot delete product with order history
);
```

Step 7 — Common Mistakes and Misconceptions

- Using CASCADE DELETE everywhere without thinking — a single top-level DELETE can wipe entire subtrees of data.
- Not indexing the foreign key column — without an index, checking/cascading child rows requires a full table scan.

- Thinking RESTRICT and NO ACTION are identical — in PostgreSQL, NO ACTION is deferred to end-of-transaction; RESTRICT is immediate.
 - Circular foreign keys — Table A FK to B, B FK to A — engine usually disallows or requires deferred checking.
 - Soft-delete pattern with CASCADE — if you use a 'deleted_at' column instead of actual deletes, CASCADE doesn't fire.
-

J

Views

What They Are · When to Use · Updatable vs Non-updatable

Step 1 — Plain English Explanation

A VIEW is a named, stored SELECT query. It looks and behaves like a table from the caller's perspective, but it has no data of its own — when you query a view, the engine runs the underlying SELECT query and returns the results. Views serve as a **security layer** (expose only certain columns to certain users), a **simplification layer** (hide complex JOINS behind a simple name), and an **abstraction layer** (if the underlying table changes, only the view definition needs updating — application code still queries the view by name). An **updatable view** allows INSERT/UPDATE/DELETE to pass through to the underlying table. A view is **non-updatable** if it contains GROUP BY, DISTINCT, aggregates, UNION, or references to multiple base tables in certain ways.

Step 2 — Real-World Analogy

A view is like a window in an office. The window shows a specific part of the room (the underlying data). Different windows can show the same room from different angles — a HR view shows only salary and department data; a finance view shows only amounts. If the furniture in the room changes (table schema), you may need to adjust the window (update the view definition), but everyone looking through a window sees the right picture without knowing the full room layout.

Step 3 — Minimal SQL Example

```
-- Create a view: simple column subset (updatable)
CREATE VIEW public_employees AS
SELECT emp_id, name, dept_id, hire_date
FROM   employees;           -- no salary exposed

-- Query the view like a table
SELECT * FROM public_employees WHERE dept_id = 3;

-- Create a non-updatable view: uses aggregates
CREATE VIEW dept_stats AS
SELECT  dept_id,
        COUNT(*) AS headcount,
        AVG(salary) AS avg_salary
FROM    employees
GROUP BY dept_id;

-- Drop a view
DROP VIEW public_employees;
```

Step 4 — Diagram — View Architecture

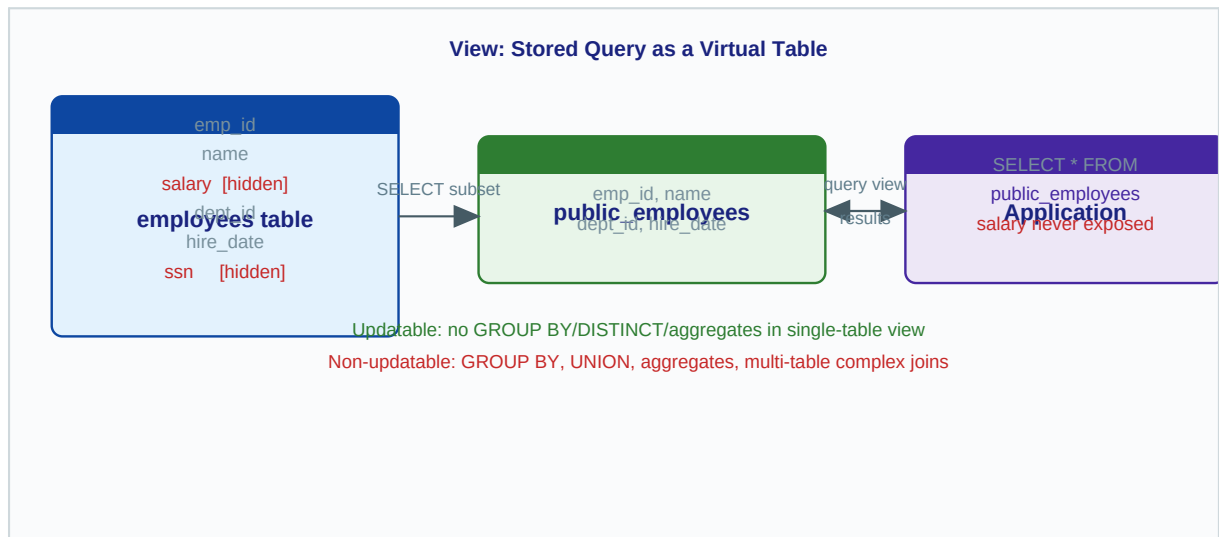


Fig J.1 — A view acts as a virtual table; the engine runs the underlying *SELECT* on query

Step 5 — Behind the Scenes

A view definition is stored in the system catalog. When you query a view, the engine performs **view expansion**: it substitutes the view's *SELECT* in place of the view name in your query, then optimises the combined query as a single statement. There is no separate caching of view results (unless you use a **MATERIALIZED VIEW**, which stores a physical snapshot of the query result and must be explicitly refreshed). Updatable views work through a set of strict rules the engine checks: single base table, no aggregates, no *DISTINCT*, no subqueries in the *SELECT* list, all *NOT NULL* columns without defaults are in the view.

Step 6 — Realistic Practical Example

```
-- Security view: HR can see salary; public cannot
CREATE VIEW employee_public AS
SELECT emp_id, name, dept_id, hire_date
FROM   employees;

CREATE VIEW employee_hr AS
SELECT emp_id, name, dept_id, hire_date, salary, manager_id
FROM   employees;

-- Convenience view: denormalised order summary
CREATE VIEW order_summary AS
SELECT  o.order_id,
        c.name           AS customer_name,
        o.order_date,
        SUM(oi.qty * oi.unit_price) AS total_amount
FROM    orders o
JOIN    customers c  ON c.customer_id = o.customer_id
JOIN    order_items oi ON oi.order_id  = o.order_id
GROUP BY o.order_id, c.name, o.order_date;
-- This view is NOT updatable due to GROUP BY
```

Step 7 — Common Mistakes and Misconceptions

- Thinking a view caches data — a regular view re-runs its query every time. For caching use **MATERIALIZED VIEW**.

- Complex views inside views (view stacking) — the engine must expand all layers; deeply nested views are hard to optimise.
 - Assuming views always improve performance — a view with joins can be slower than a direct query if the optimiser cannot push down filters.
 - Updating a view that joins two tables — UPDATE through a join view is not supported in most engines; use INSTEAD OF triggers.
 - Forgetting to add WITH CHECK OPTION — without it, INSERTs/UPDATEs through a view can create rows invisible to the view.
-

SQL Fundamentals — Quick Reference Cheat Sheet

Statement / Concept	Syntax / Rule	Key Gotcha
CREATE TABLE	CREATE TABLE t (col TYPE CONSTRAINT, ...);	PRIMARY KEY auto-creates B-tree index
ALTER TABLE	ALTER TABLE t ADD COLUMN col TYPE;	On large tables this can rewrite all rows
INSERT	INSERT INTO t (cols) VALUES (...);	Always list column names explicitly
UPDATE	UPDATE t SET col=val WHERE condition;	Omit WHERE = update every row
DELETE	DELETE FROM t WHERE condition;	Omit WHERE = delete every row
SELECT execution order	FROM > JOIN > WHERE > GROUP BY > HAVING > SELECT > ORDER BY > LIMIT	Alias in SELECT not visible in WHERE
INNER JOIN	JOIN t2 ON t1.id = t2.fk	Omit ON = cartesian product
LEFT JOIN	LEFT JOIN t2 ON ...	NULL in right cols = no match
GROUP BY	SELECT col, AGG() FROM t GROUP BY col	All non-agg SELECT cols must be in GROUP BY
HAVING	HAVING AGG() > value	HAVING filters groups; WHERE filters rows
NULL check	WHERE col IS NULL / IS NOT NULL	col = NULL always returns UNKNOWN
COALESCE	COALESCE(col, default_val)	Returns first non-NULL value in list
EXISTS	WHERE EXISTS (SELECT 1 FROM ...)	Stops at first match; safe with NULLs
NOT IN + NULL	Avoid NOT IN (subquery with NULLs)	Returns 0 rows if subquery has any NULL
CASCADE DELETE	ON DELETE CASCADE	Recursively deletes child rows
View	CREATE VIEW v AS SELECT ...;	Non-updatable if GROUP BY/aggregates present